Chapter 6. Neural Networks and In-Context Learning for Tabular Data

Predictive Modeling, Lecture 5 (MLP, embeddings, backpropagation, Adam, regularization, TabPFN, TabICL)

Contents

1 When Tree Boosting Is Not Enough **2**

2 The MLP as a Composite Nonlinear Function **2**

2.1 One hidden layer 2

3 Neural-Network Inputs Are Numbers **3**

4 Categorical Embeddings Are Learned From Prediction Error **3**

5 Training an MLP: Forward, Backward, Update **4**

5.1 Minibatches and Adam 4

6 The Core Difficulty of Tabular MLPs: Overfitting **4**

7 Trees and MLPs Have Different Inductive Biases **5**

8 Another Route: In-Context Learning **5**

8.1 Refitting versus conditioning 5

8.2 TabPFN pre-trains the learning algorithm itself 6

8.3 TabICL summarizes rows first, then learns in context 6

9 Summary **7**

10 Selected exercises **7**

1 When Tree Boosting Is Not Enough

Limitation of tree boosting		What neural networks offer
finds splits on given features but does not learn the representation of features	$\longrightarrow$	learns representations needed for prediction through **embeddings** and **hidden layers**
smooth relations may need many splits and trees	$\longrightarrow$	combines several features continuously to represent complex relations
text, image, time series must first be summarized into tabular features	$\longrightarrow$	different input types can be learned in one model
usually re-learns a model from scratch for each new data set	$\longrightarrow$	pre-training and in-context learning reuse information from other problems

$$\underbrace{\text{fixed features} \longrightarrow \text{split learning}}_{\text{tree boosting}} \qquad \underbrace{\text{representation learning} + \text{prediction learning}}_{\text{neural network}}$$

2 The MLP as a Composite Nonlinear Function

2.1 One hidden layer

$$z^{(1)} = W^{(1)}x + b^{(1)} \quad \text{weighted sums of the raw features,}$$
$$a^{(1)} = \mathrm{ReLU}(z^{(1)}) \quad \text{hidden representation after activation,}$$
$$\hat{y} = g(W^{(2)}a^{(1)} + b^{(2)}) \quad \text{final prediction from the hidden representation.}$$

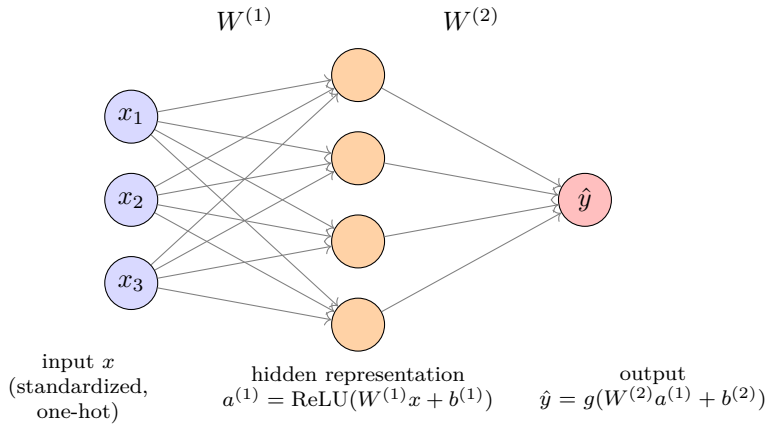


Figure 1: A one-hidden-layer MLP; g is identity, sigmoid or softmax. Backpropagation sends $\hat{y} - y$ back along the edges.

Hidden layer: a one-hidden-layer ReLU network is continuous piecewise linear, cut by the hyperplanes $\{w_k^{(1)\top}x + b_k^{(1)} = 0\}$; with enough units it approximates any continuous function on a compact set (universal approximation; Exercise 6.2).

Output layer by target:

Target	Output g	Loss	Gradient w.r.t. the output score
regression	identity $g(u) = u$	$\frac{1}{2}(y - \hat{y})^2$	$\hat{y} - y$
binary	sigmoid	log loss	$\hat{p} - y$
multiclass	softmax $p_k = e^{u_k} / \sum_l e^{u_l}$	cross-entropy $-\sum_k y_k \log p_k$	$p - y$

Proof of the softmax gradient [Supplement]. $\partial \log p_k / \partial u_m = \mathbb{I}(k = m) - p_m$, so $\partial(-\sum_k y_k \log p_k)/\partial u_m = -y_m + p_m \sum_k y_k = p_m - y_m$. ■

3 Neural-Network Inputs Are Numbers

One customer record:

Raw data	Conversion to numbers	Input
age 52	standardize with mean and SD (training data)	0.8
occupation: teacher	one-hot vector	$(0, 1, 0)$
income: missing	median imputation and missing indicator	$(40, 1)$

Input $(0.8, 0, 1, 0, 40, 1)$: the unscaled 40 dominates the weighted sums, so standardize (training statistics only).

4 Categorical Embeddings Are Learned From Prediction Error

For K occupations learn $E \in \mathbb{R}^{K \times d}$, $d \ll K$; teacher $= 1$ fetches row 1:

$$e_{\text{teacher}} = E[1, :] = (0.7, -0.2, 0.4), \qquad o_{\text{teacher}}^\top E = E[1, :],$$

a lookup equal to one-hot times a weight matrix.

Algorithm: how an embedding is learned

1. Initialize the entries of E with small random numbers.
2. Use the row of the observed category as network input.
3. Compute the error between truth and prediction.
4. Update, by backpropagation, only the row that was used:

$$E[1, :] \leftarrow E[1, :] - \eta \frac{\partial \ell}{\partial E[1, :]}.$$

Theorem: the gradient touches only the used row

If $e = o_k^\top E$ enters the network, then $\dfrac{\partial \ell}{\partial E} = o_k \left(\dfrac{\partial \ell}{\partial e}\right)^\top$, a $K \times d$ matrix whose only nonzero row is row k.

Proof [Supplement]. $e_m = \sum_l o_{k,l} E_{lm} = E_{km}$, so $\partial e_m / \partial E_{lm'} = \mathbb{I}(l = k)\mathbb{I}(m = m')$; apply the chain rule. ■

E is learned with the other weights; similar categories get nearby rows, rare ones stay noisy.

5 Training an MLP: Forward, Backward, Update

Algorithm: minibatch training

Initialize the weights θ. For each epoch and each minibatch B_t:

1. **Forward**: compute predictions and the average loss $\widehat{R}_{B_t}(\theta) = \frac{1}{|B_t|}\sum_{i\in B_t} L\{y_i, f_\theta(x_i)\}$.
2. **Backward**: compute the gradient $g_t = \nabla_\theta \widehat{R}_{B_t}(\theta)$.
3. **Update**: move the weights in the opposite direction, $\theta \leftarrow \theta - \eta g_t$.

At the end of each epoch record the validation loss; at the end, restore the weights with the lowest validation loss.

Backward pass: chain rule $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial w}$.

Derivation: backpropagation for one hidden layer

For $z^{(1)} = W^{(1)}x + b^{(1)}$, $a^{(1)} = \mathrm{ReLU}(z^{(1)})$, $\hat{y} = W^{(2)}a^{(1)} + b^{(2)}$, $L = \frac{1}{2}\|y - \hat{y}\|^2$:

$$\delta^{(2)} = \hat{y} - y, \quad \frac{\partial L}{\partial W^{(2)}} = \delta^{(2)}a^{(1)\top}, \quad \frac{\partial L}{\partial b^{(2)}} = \delta^{(2)},$$

$$\delta^{(1)} = \left(W^{(2)\top}\delta^{(2)}\right) \odot \mathbb{I}\{z^{(1)} > 0\}, \quad \frac{\partial L}{\partial W^{(1)}} = \delta^{(1)}x^\top, \quad \frac{\partial L}{\partial b^{(1)}} = \delta^{(1)}.$$

Reusing δ makes all gradients cost about one forward pass; units with $z \le 0$ pass no gradient ("dead" ReLU).

5.1 Minibatches and Adam

For uniform batches $\mathbb{E}[g_t] = \nabla\widehat{R}(\theta)$ at cost $O(|B_t|)$. Adam adapts per-parameter steps:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2,$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \ \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad \theta \leftarrow \theta - \eta\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon},$$

($\beta_1 = 0.9$, $\beta_2 = 0.999$). Bias correction undoes $m_0 = v_0 = 0$: constant mean μ gives $\mathbb{E}m_t = (1-\beta_1^t)\mu$. $\hat{m}/\sqrt{\hat{v}}$ is roughly scale-free. Minibatch, Adam: *optimization* devices.

6 The Core Difficulty of Tabular MLPs: Overfitting

training loss keeps decreasing + validation loss rises again $\Longrightarrow$ overfitting.

Device	Role	Signal to check
small model	limit depth and width, fewer parameters	does even the training loss stay high (underfitting)?
weight decay	penalize large weights	does the train–validation gap shrink?
dropout	randomly switch off some hidden units during training	does validation improve, or does it overfit?
early stopping	restore the weights of the epoch with minimal validation loss	from when does validation loss rise again?

Theorem: weight decay and dropout as regularizers

(i) Adding $\frac{\lambda}{2}\|\theta\|^2$ to the loss changes the gradient step to $\theta \leftarrow (1-\eta\lambda)\theta - \eta g_t$: every step multiplicatively shrinks the weights (Ridge, Ch. 3).
(ii) Inverted dropout replaces a unit a by $\tilde{a} = a\,m/(1-q)$ with $m \sim \text{Bernoulli}(1-q)$, so $\mathbb{E}\tilde{a} = a$. For a linear model $\hat{y} = \beta^\top \tilde{x}$ with dropout on the inputs,

$$\mathbb{E}_m[(y - \beta^\top \tilde{x})^2] = (y - \beta^\top x)^2 + \frac{q}{1-q}\sum_j \beta_j^2 x_j^2,$$

i.e. dropout behaves like a data-dependent Ridge penalty.

Proof [Supplement]. (i) $\nabla(\widehat{R} + \frac{\lambda}{2}\|\theta\|^2) = g_t + \lambda\theta$.

(ii) $\mathbb{E}[\beta^\top \tilde{x}] = \beta^\top x$, so $\mathbb{E}(y - \beta^\top \tilde{x})^2 = (y - \beta^\top x)^2 + \text{Var}(\beta^\top \tilde{x})$, and independent m_j with $\text{Var}\{m_j/(1-q)\} = q/(1-q)$ give $\text{Var}(\beta^\top \tilde{x}) = \sum_j \beta_j^2 x_j^2\, q/(1-q)$. ■

Weight decay, dropout, early stopping: *generalization* devices; early stopping $\approx L_2$ (Exercise 6.3).

7 Trees and MLPs Have Different Inductive Biases

	Tree boosting	MLP
representation	piecewise functions built from stacked feature thresholds	continuous hidden representation from weighted sums and activations
preprocessing	relatively insensitive to the scale of numerical features	sensitive to scaling, missing values, categorical encoding
data size	strong baseline on small/medium tabular data	benefits from sufficient data
computation	many efficient CPU implementations	suited to minibatches and GPU matrix operations

On small tabular data MLPs have many parameters per observation, must build thresholds from many neurons, and depend on encoding; heterogeneous, irregular, partly uninformative columns favor trees.

8 Another Route: In-Context Learning

Motivation. A model pretrained on synthetic tables predicts from labeled examples given as input.

8.1 Refitting versus conditioning

MLP and GBM: fit anew	TabPFN, TabICL: use as context
$\mathcal{D}_{\text{train}} \xrightarrow{\text{fit}} \theta_{\mathcal{D}} \xrightarrow{x_q} \hat{y}_q$	$\Big(\underbrace{\{(x_i, y_i)\}_{i \in C}}_{\text{labeled cases}}, \underbrace{x_q}_{\text{new case}}\Big) \xrightarrow{\text{fixed pre-trained model}} \hat{y}_q$
a new model is learned for each new data set	uses a model pre-trained on many tabular tasks
weights or trees are updated from the training data	predicts usually without re-training weights
estimates can be unstable with small data	

In-context learning keeps the model fixed and feeds the data as input.

8.2 TabPFN pre-trains the learning algorithm itself

Algorithm: TabPFN

Pre-training. (1) Generate many synthetic tables from diverse generating structures (prior over structural causal models, MLPs, trees, noise, ...).
(2) Split each table into rows with visible labels and rows with hidden labels.
(3) Train to predict the hidden labels from the visible rows.
(4) Repeat over many tabular tasks to learn Θ.

Prediction on a new data set. $(X_{\text{train}}, y_{\text{train}}, x_q) \xrightarrow{\text{fixed } \Phi_\Theta} \hat{p}(y_q)$: a Transformer reads the whole table with unchanged weights.

Theorem: what prior-fitted training approximates

Let tasks $\phi \sim \pi$, and given ϕ draw a data set $\mathcal{D}$ and a query (x_q, y_q). If q_Θ ranges over all conditional distributions, the minimizer of the pre-training loss

$$\mathbb{E}_{\phi,\mathcal{D},x_q,y_q}\big[-\log q_\Theta(y_q \mid x_q, \mathcal{D})\big]$$

is the **posterior predictive distribution** $p(y_q \mid x_q, \mathcal{D}) = \int p(y_q \mid x_q, \phi)\, p(\phi \mid \mathcal{D})\, d\phi$.

Proof [Supplement]. Conditioning on $(\mathcal{D}, x_q)$, the loss is $\mathbb{E}_{\mathcal{D},x_q}[\mathrm{KL}(p(\cdot \mid x_q, \mathcal{D}) \,\|\, q_\Theta(\cdot \mid x_q, \mathcal{D}))] + \mathbb{E}[H(p(\cdot \mid x_q, \mathcal{D}))]$; the entropy is free of Θ and $\mathrm{KL} \geq 0$ with equality iff $q_\Theta = p(\cdot \mid x_q, \mathcal{D})$. Finally $p(y_q \mid x_q, \mathcal{D}) = \int p(y_q \mid x_q, \phi) p(\phi \mid x_q, \mathcal{D}) d\phi$ with $p(\phi \mid x_q, \mathcal{D}) = p(\phi \mid \mathcal{D})$. ■

TabPFN approximates Bayesian prediction under its synthetic prior, so quality depends on prior coverage.

8.3 TabICL summarizes rows first, then learns in context

Let the input table be $X = (x_{ij}) \in \mathbb{R}^{n \times m}$ with n rows and m features.

1. **Column-wise**: $(x_{1j}, \ldots, x_{nj}) \longrightarrow (e_{1j}, \ldots, e_{nj})$ (50 means different things in age and income).
2. **Row-wise**: $(e_{i1}, \ldots, e_{im}) \longrightarrow h_i \in \mathbb{R}^d$, fixed length for any m.
3. **In-context**: $(\{(h_i, y_i)\}_{i=1}^n, h_q) \xrightarrow{\text{pre-trained TabICL}} \hat{y}_q$.

Row compression lets the in-context step scale to more rows.

Remark: cautions for in-context models

Context = training rows of the current split only (else Ch. 1 leakage). Limits: context length, prior mismatch, prediction-time compute.

9 Summary

Data condition	Candidates to compare first	What to check
small/medium mixed-type data	regularized regression, CatBoost, LightGBM	is performance stable across splits?
enough data or useful categorical embeddings	tree boosting and a small MLP	were preprocessing and overfitting controlled?
pre-training and ICL usable	add TabPFN/TabICL to the existing baselines	do problem type and data size fit, within the compute budget?

MLP: preprocessing and embeddings to a hidden representation; minibatch/Adam optimize, weight decay/dropout/early stopping generalize. **TabPFN, TabICL**: labeled cases as context; sensitive to context length, prior and leakage.

10 Selected exercises

Motivation. Random initialization, ReLU representation in one dimension, early stopping as Ridge.

Exercise 6.1 (Identical hidden units stay identical)

Consider the one-hidden-layer network of the backpropagation box with scalar output $\hat{y} = w^{(2)\top} a^{(1)} + b^{(2)}$ and any differentiable loss $L(y, \hat{y})$, trained by (minibatch) gradient descent; for ReLU use the convention $\mathbb{I}\{z > 0\}$ for its derivative. Suppose that at $t = 0$ hidden units j and k have the same incoming weights and bias, $W^{(1)}_{j\cdot} = W^{(1)}_{k\cdot}$, $b^{(1)}_j = b^{(1)}_k$, and the same outgoing weight, $w^{(2)}_j = w^{(2)}_k$. Show that these equalities hold at every step t.

Solution [Supplement].

Step 1. If the equalities hold at step t, then for every input x, $z_j = z_k$ and $a_j = a_k$.

Step 2. With $\delta^{(2)} = \partial L/\partial \hat{y}$: $\partial L/\partial w^{(2)}_j = \delta^{(2)} a_j = \partial L/\partial w^{(2)}_k$; $\delta^{(1)}_j = w^{(2)}_j \delta^{(2)} \mathbb{I}\{z_j > 0\} = \delta^{(1)}_k$, so $\partial L/\partial W^{(1)}_{j\cdot} = \delta^{(1)}_j x^\top = \partial L/\partial W^{(1)}_{k\cdot}$ and $\partial L/\partial b^{(1)}_j = \partial L/\partial b^{(1)}_k$; minibatch averaging preserves this.

Step 3. Equal parameters get equal updates (also under Adam, which is coordinate-wise); induct on t.

Equal starts collapse the width to one unit; random initialization breaks the symmetry. ■

Exercise 6.2 (One hidden ReLU layer and piecewise-linear functions)

(a) Let $f : [a, b] \to \mathbb{R}$ be continuous and linear on each $[t_{k-1}, t_k]$, $a = t_0 < t_1 < \cdots < t_K = b$, with slope m_k there. Show that for $x \in [a, b]$

$$f(x) = f(a) + m_1 \,\mathrm{ReLU}(x - a) + \sum_{k=1}^{K-1} (m_{k+1} - m_k) \,\mathrm{ReLU}(x - t_k),$$

so a network with K hidden ReLU units represents f exactly on $[a, b]$.
(b) Deduce that every continuous f on $[a, b]$ is a uniform limit of such networks.

Solution [Supplement].

(a)

Step 1. Let g be the right side. On $[t_{k-1}, t_k]$, $\mathrm{ReLU}(x - t_l) = 0$ for $l \geq k$, $= x - t_l$ for $l \leq k-1$, and $\mathrm{ReLU}(x - a) = x - a$: g is continuous and piecewise linear.

Step 2. Slope on (t_{k-1}, t_k): $m_1 + \sum_{l=1}^{k-1}(m_{l+1} - m_l) = m_k$; and $g(a) = f(a)$.

Step 3. $f - g$ is continuous with slope 0 on every piece and 0 at a, so $f = g$. Weights: input 1, biases $-a, -t_1, \ldots, -t_{K-1}$, outputs $m_1, m_2 - m_1, \ldots, m_K - m_{K-1}$, output bias $f(a)$.

(b)

Step 4. Given $\varepsilon > 0$, uniform continuity gives $h > 0$ with $|f(x) - f(x')| < \varepsilon$ for $|x - x'| \leq h$; let ℓ interpolate f linearly on knots of mesh $\leq h$.

Step 5. On $[t_{k-1}, t_k]$, $\ell(x)$ is a convex combination of $f(t_{k-1}), f(t_k)$, both within ε of $f(x)$, so $|\ell - f| < \varepsilon$; ℓ is a network by (a). ■

Exercise 6.3 (Early stopping and Ridge)

Let $L(\theta) = \frac{1}{2}\|\mathbf{y} - X\theta\|^2$ with thin SVD $X = UDV^\top$, $d_1 \geq \cdots \geq d_r > 0$, and run gradient descent $\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$ from $\theta_0 = \mathbf{0}$ with $0 < \eta < 1/d_1^2$.
(a) Show that $\theta_t = V\mathrm{diag}(\phi_t(d_j)/d_j)U^\top \mathbf{y}$ with $\phi_t(d) = 1 - (1 - \eta d^2)^t$.
(b) The Ridge solution of $\frac{1}{2}\|\mathbf{y} - X\theta\|^2 + \frac{\lambda}{2}\|\theta\|^2$ is $V\mathrm{diag}(\psi_\lambda(d_j)/d_j)U^\top \mathbf{y}$ with $\psi_\lambda(d) = d^2/(d^2 + \lambda)$ (SVD form of Ridge). Show that for $\lambda = 1/(\eta t)$

$$\psi_\lambda(d) \leq \phi_t(d) \leq 2\,\psi_\lambda(d) \qquad \text{for all } d \in (0, d_1].$$

Solution [Supplement].

(a)

Step 1. $\theta_{t+1} = (I - \eta X^\top X)\theta_t + \eta X^\top \mathbf{y}$ with $X^\top \mathbf{y} = VDU^\top \mathbf{y}$, $X^\top X = VD^2V^\top$ and $\theta_0 = \mathbf{0}$, so $\theta_t = Vc_t$.

Step 2. $c_{t+1,j} = (1 - \eta d_j^2)c_{t,j} + \eta d_j\, u_j^\top \mathbf{y}$ has fixed point $c_j^\star = u_j^\top \mathbf{y}/d_j$, and $c_{t,j} - c_j^\star = (1 - \eta d_j^2)^t(0 - c_j^\star)$, so $c_{t,j} = \phi_t(d_j)\, u_j^\top \mathbf{y}/d_j$.

(b)

Step 3. Put $u = \eta d^2 \in (0, 1)$. Then $\psi_\lambda(d) = \eta t d^2/(\eta t d^2 + 1) = ut/(1 + ut)$ and $\phi_t(d) = 1 - (1 - u)^t$.

Step 4. (Lower bound.) $(1 - u)^t \leq e^{-ut} \leq 1/(1 + ut)$, so $\phi_t \geq 1 - 1/(1 + ut) = \psi_\lambda$.

Step 5. (Upper bound.) Bernoulli: $(1 - u)^t \geq 1 - ut$, so $\phi_t \leq \min(1, ut)$. If $ut \leq 1$, $\psi_\lambda \geq ut/2$; if $ut > 1$, $\psi_\lambda > \frac{1}{2}$. Either way $\min(1, ut) \leq 2\psi_\lambda$.

More steps correspond to a smaller penalty. ■